

Remaining Useful Life Estimation on NASA CMAPSS FD001: Tabular Baselines and a One-Dimensional Convolutional Network

Nikhil Kumar Gane

Spring 2026

Abstract

Accurate estimation of *remaining useful life* (RUL) for turbofan engines supports condition-based maintenance and reduces unplanned downtime. This project uses the NASA CMAPSS turbofan degradation benchmark (subset FD001), where each engine yields a multivariate time series of sensor readings and operating settings until failure (training) or censoring before failure (test). We construct piecewise-linear RUL targets capped at 125 cycles, hold out 20% of training engines for validation to avoid temporal leakage, and compare three regressors: linear regression, a random forest, and a small one-dimensional convolutional neural network (1D CNN) applied to sliding windows of length 30. On the official test split evaluated at the last observed cycle per engine, the random forest achieves the best test MAE (12.01) and RMSE (17.05), while the CNN improves over linear models on validation but underperforms the forest on test under our training protocol, indicating room for temporal-model tuning. Code reproduces preprocessing, training, and metrics in a single Python entry point.

1 Introduction

Prognostics and health management (PHM) aims to predict how much useful life remains in a component or system from operational data. For commercial and military aircraft engines, reliable RUL estimates support scheduling of inspections and part replacement, improve safety margins, and can lower lifecycle cost compared with fixed-interval maintenance [1]. The learning problem is challenging: degradation is latent, sensors are noisy, and different units start with different unknown initial wear.

The NASA CMAPSS data simulator provides controlled multivariate time series labeled by run-to-failure trajectories in training and by censored trajectories with held-out RUL labels in test [1]. Subset FD001 corresponds to a single operating condition and a single fault mode (HPC degradation), which simplifies modeling while preserving realistic sensor structure.

In this project we treat RUL estimation as supervised regression. For each time index in training we derive a target RUL from the known failure time of that engine, apply a standard piecewise cap on large RUL values to reduce skew, and exclude identifiers that are not physically meaningful as direct inputs (engine index and cycle counter). We compare classical tabular models that use only the current snapshot of settings and sensors against a temporal model that stacks the past 30 cycles into a tensor for a 1D CNN. Our goals are: (1) to establish reproducible baselines with clear evaluation protocol, (2) to quantify whether a lightweight temporal model offers gains over the same features without hand-crafted

trend statistics, and (3) to document failure modes (e.g., generalization gap for the CNN on test) for future improvement.

Contributions. We provide an end-to-end FD001 pipeline (loading, label construction, engine-level splits, training, and evaluation on the last test snapshot per engine), report validation and test MAE/RMSE for three models, and discuss why the random forest remains strongest under the current feature set and CNN training budget.

1.1 Problem formulation

Let $\mathbf{x}_{i,t} \in \mathbb{R}^d$ denote the vector of settings and sensors for unit i at cycle t after preprocessing (here $d=24$). The training set consists of pairs $(\mathbf{x}_{i,t}, y_{i,t})$ for all observed (i, t) with targets from Eq. (1). The learning objective is to find a function f minimizing empirical risk under square or absolute loss, e.g.,

$$\hat{f} \in \arg \min_{f \in \mathcal{F}} \frac{1}{N} \sum_{i,t} \ell(f(\mathbf{z}_{i,t}), y_{i,t}), \quad (1)$$

where $\mathbf{z}_{i,t}$ is either $\mathbf{x}_{i,t}$ for tabular models or a window tensor for the CNN, and ℓ is the squared error in training for the neural model (MAE is reported for interpretability at evaluation). Test-time prediction focuses on the last time $\tilde{T}_i$ for each test engine: we report $\ell_{\text{MAE}} = \frac{1}{N_{\text{test}}} \sum_i |\hat{y}_{i,\tilde{T}_i} - \tilde{y}_i|$ and the corresponding RMSE.

1.2 Scope and non-goals

We restrict attention to FD001 to match the implemented codebase and to avoid mixture-of-regimes complications present in FD002 and FD004 (multiple flight conditions) and dual-fault modes in FD003/FD004 [1]. We do not optimize fleet-level decision policies (e.g., maintenance scheduling under cost asymmetry); we only estimate scalar RUL.

2 Related Work

Turbofan simulation and benchmarks. Saxena et al. [1] introduced the CMAPSS simulator and the FD001–FD004 subsets that have since become standard benchmarks for prognostics competitions and algorithm comparison. The dataset design—multivariate snapshots per cycle, three operational settings, and many redundant or near-constant sensors—motivates careful preprocessing and sensor selection, which many later works address explicitly.

Degradation modeling and RUL targets. Piecewise-linear RUL caps (e.g., 125 cycles for FD001) are widely used to prevent models from spending capacity on distinguishing very large RUL values that are less operationally critical and to stabilize training for neural models [2]. We adopt the same convention so our scores are comparable to common practice in the literature.

Neural sequence models for RUL. Recurrent architectures, including early recurrent networks applied to health monitoring windows [4] and later LSTM-focused studies [3], model temporal dependence through hidden state transitions. Convolutional models treat the time axis as a one-dimensional signal: multi-channel convolutions over sensor trajectories learn local temporal patterns and can be more

parallelizable than RNNs [5]. Hybrid CNN–LSTM stacks appear frequently in the CMAPSS literature to combine local motif extraction with longer-range pooling. Our CNN is deliberately small and omits an LSTM stage to isolate the contribution of convolutional temporal filtering under a fixed compute budget.

Nonparametric tabular baselines. Random forests and gradient boosted trees remain strong baselines on engineered tabular features and sometimes match or exceed deep models when temporal structure is weakly exploited or when training data are limited [6]. Our results align with this pattern when the CNN is trained with a simple protocol.

Contrast to our work. We do not introduce a new architecture; instead we implement transparent baselines and a first 1D CNN with documented splits and metrics. Relative to LSTM-centric studies, our CNN uses the same raw per-cycle measurements in a window without extra hand-crafted health indices. Relative to industrial-scale pipelines, we omit sensor selection ablations and the NASA asymmetric scoring function, which we note as future work.

3 Methods

3.1 Dataset and notation

Table 1 summarizes the FD001 split shipped with CMAPSS. Each file row is one engine i at cycle t , with three operational settings and 21 sensor channels used in our code (constant or uninformative sensors in the raw 26-channel layout are still present numerically but not dropped in this baseline). Training engines run until failure; test engines stop before failure and each has a scalar true RUL after the last recorded cycle, provided in `RUL_FD001.txt` in engine order [1].

Table 1: NASA CMAPSS FD001 subset used in this project (from dataset documentation).

Quantity	Value
Training trajectories (engines)	100
Test trajectories (engines)	100
Operating conditions	1 (sea level)
Fault modes	1 (HPC degradation)
Raw columns per row	26 (unit, time, 3 settings, 21 sensors)

Parsing. Text files are whitespace-separated; trailing spaces occasionally create empty trailing columns when parsed naively. Our loader drops all-null columns and asserts the expected width after cleanup, then assigns consistent column names (`engine_id`, `cycle`, `setting_1..3`, `sensor_1..21`).

Test labels. For test engine i , let c_i be the last recorded cycle in `test_FD001.txt`. The label file provides ρ_i such that the (unobserved) failure cycle would be $c_i + \rho_i$. Predictions are evaluated only at $t = c_i$.

3.2 Label construction

For training row (i, t) let T_i be the last observed cycle of engine i in the training set (failure time). The raw remaining life is $R_{i,t}^{\text{raw}} = T_i - t$. We define the training target

$$y_{i,t} = \min(R_{i,t}^{\text{raw}}, R_{\max}), \quad R_{\max} = 125. \quad (2)$$

For test evaluation at the final cycle $\tilde{T}_i$ of engine i , the provided label ρ_i is compared using the same cap on the label side: $\tilde{y}_i = \min(\rho_i, R_{\max})$, so training and scoring use consistent piecewise linear semantics.

3.3 Temporal leakage and splits

Random row splits would leak information across cycles of the same engine. We split *engine identifiers*: 80 training engines and 20 validation engines (20% holdout, seed 42) for model selection and reporting validation metrics. For final test numbers, sklearn baselines are refit on *all* training engines before predicting the last snapshot of each test engine.

3.4 Tabular baselines

Features. At each time step we use the three settings and 21 sensors; we exclude `engine_id` and `cycle` from X to avoid trivial dependence on unit index and absolute timeline.

Preprocessing. We use median imputation (defensive; FD001 has few missing values) followed by per-feature standardization inside a `sklearn` pipeline.

Models. *Linear regression* (ordinary least squares on standardized features) provides a simple lower bound. *Random forest* regression uses 200 trees, default depth, and parallel fitting (`n_jobs=-1`), which captures nonlinear interactions and heterogeneous sensor importance without explicit temporal structure. Each tree fits piecewise-constant rules on random feature subsets; ensembling reduces variance relative to a single deep tree [6].

3.5 Temporal model: 1D CNN on sliding windows

For each engine we form windows of length $L = 30$ ending at cycle t . The input tensor has shape (C, L) with $C = 24$ channels (settings plus sensors). A small network stacks two Conv1d layers with kernel size 3, padding 1, each with 64 output channels (the first maps from $C=24$ input channels to 64; the second maps $64 \rightarrow 64$), ReLU activations, global adaptive average pooling over time, and a linear head mapping the 64-dimensional pooled vector to one scalar RUL prediction. Parameters are optimized with Adam (learning rate 10^{-3}) on mean squared error in mini-batches of 256 sequences.

Training protocol. For validation metrics, we train with early stopping (patience 12 epochs) monitoring MSE on sequences from held-out engines. For the final test evaluation the CNN is trained on all training sequences for a fixed number of epochs on the full training set (no engine-held-out inner validation in that refit) to avoid row-level leakage from splitting windows that belong to the same engine; this is a pragmatic choice that may increase overfitting risk and partly explains test performance.

Scaling for the CNN. A `StandardScaler` is fit on *numpy* matrices of training-engine rows only (to avoid feature-name warnings in `sklearn`) and applied to construct normalized windows. For test engines

with fewer than L recorded cycles, we front-pad by repeating the first observed row so a window always exists; this is a limitation for very short tests.

3.6 Repository layout and reproducibility hooks

The code is organized as: `cmapss_utils.py` (paths, loading, capped targets, test alignment), `run_project.py` (end-to-end experiment driver), and `fd001_baseline_workflow.py` (smaller educational script that reuses the utilities). Dependencies are pinned loosely in `requirements.txt` (`numpy`, `pandas`, `scikit-learn`, `torch`). Running the driver from the repository root regenerates `REPORT.md` and `results/metrics.json`, which are the sources for the tables in this document.

3.7 Evaluation metrics

We report mean absolute error (MAE) and root mean squared error (RMSE) between predictions and capped targets. All scores use the same unit (cycles).

4 Experimental Results

4.1 Implementation and reproducibility

Experiments are implemented in Python 3 using `pandas`, `scikit-learn`, and `torch`. Running `python run_project.py` from the repository root regenerates metrics and writes `REPORT.md` and `results/metrics.json`. Random seed 42 controls the engine split and forest/CNN initialization. Experiments were timed on a typical laptop CPU (GPU optional); end-to-end runtime is on the order of a few minutes dominated by CNN epochs.

4.2 Alignment between report and automated outputs

All numeric entries in Tables 2–3 match the last automated run recorded in `REPORT.md` (rounded to two decimals). If graders re-execute the code, tiny floating-point differences may appear depending on BLAS/threading settings; the repository documents the intended protocol first and foremost.

4.3 Validation performance

Table 2 summarizes performance on held-out training engines (tabular models on per-row snapshots; CNN on sequence samples from held-out engines). The random forest achieves the lowest RMSE, followed by the CNN and then linear regression.

Table 2: Validation MAE and RMSE (capped RUL targets).

Model	MAE	RMSE
Linear regression	16.48	20.41
Random forest	12.56	17.17
1D CNN ($L=30$)	13.72	17.95

4.4 Test performance

Table 3 reports results on the 100 FD001 test engines. Each model predicts RUL at the last observed cycle; errors are computed against $\min(\rho_i, 125)$.

Table 3: Test MAE and RMSE (last snapshot per engine, capped true RUL).

Model	MAE	RMSE
Linear regression	16.57	20.83
Random forest	12.01	17.05
1D CNN ($L=30$)	15.09	20.28

4.5 Analysis

The random forest generalizes best on the test split under our protocol. The CNN improves over the linear baseline on validation, suggesting some temporal structure is learnable, but test MAE/RMSE regress toward weaker performance than the forest. Plausible causes include: limited model capacity; fixed window length not tuned for FD001; front-padding for short tests; MSE training without asymmetric loss; and the final CNN refit on all sequences without engine-aware early stopping. Sensor selection and removal of near-constant channels, which many CMAPSS papers emphasize, are not yet applied and could lift all models.

Statistical note (course requirement). COMS5740 groups are asked for formal statistical comparison of methods. This manuscript targets COMS4740-style reporting with a single official partition; we do not report p -values. A straightforward extension is paired bootstrap over test engines (resampling engines with replacement and recomputing MAE differences) or repeated random engine splits on the training pool to quantify variance [7].

4.6 Ablation and risk checklist (informal)

We did not sweep window lengths L , forest depth, or CNN width; we did not evaluate NASA’s asymmetric score; we did not train on FD002–FD004 multi-condition data. Each item is a documented limitation rather than an unreported negative result.

4.7 Ethics, data use, and limitations

The CMAPSS data are simulated rather than collected from operational aircraft; nevertheless, careless deployment of prognostic models on real fleets could create false confidence. Our work is research-grade: we do not claim certification for safety-critical decisions. Labels are correct only under simulator assumptions; transfer to real engines would require domain validation, drift monitoring, and governance review.

4.8 Threats to validity and external validity

Construct validity. Using MAE and RMSE on capped RUL aligns with many published CMAPSS studies but differs from the PHM08 challenge scoring function, which penalizes late predictions more

heavily than early ones. A model could rank differently under that metric.

Internal validity. The CNN’s final training stage uses all engines without an engine-level inner validation loop for model selection, which complicates claims that validation performance predicts test performance for that model family.

External validity. FD001 involves a single operating condition; conclusions need not transfer to FD002/FD004-style mixtures without retraining or domain adaptation. Similarly, removing near-constant sensors (a common preprocessing step) changes the feature distribution and can alter relative model rankings.

4.9 Practical takeaways for maintenance analytics

Even when temporal models are intellectually attractive, strong tabular baselines remain mandatory benchmarks in industrial prognostics projects. Random forests are comparatively easy to tune, expose feature importances for subject-matter review, and often perform well on per-cycle snapshots when degradation correlates with instantaneous sensor state. Neural temporal models may still win after investment in windowing, architecture search, and careful prevention of leakage, but that investment should be justified by measured gains on the target metric and deployment constraints (latency, explainability).

5 Conclusion

We built a reproducible FD001 RUL pipeline with piecewise capped labels, engine-level validation, linear and random forest baselines, and a 1D CNN over length-30 windows. The random forest remains the strongest model on test MAE/RMSE in our setting, while the CNN demonstrates partial benefit on validation but not a consistent win on the held-out test engines. Future work includes window and architecture search, sensor filtering, recurrent or attention temporal encoders, multi-dataset training, statistical confidence intervals over engines, and alignment with competition scoring functions.

A Command-line reproduction checklist

1. Obtain CMAPSSData/ with `train_FD001.txt`, `test_FD001.txt`, and `RUL_FD001.txt`.
2. Create a Python environment and run `pip install -r requirements.txt`.
3. From the repository root, run `python run_project.py`; confirm `REPORT.md` updates.
4. Optionally run `python fd001_baseline_workflow.py` for the shorter baseline-only narrative print-out.

Table 4: Key hyperparameters (defaults unless noted).

Component	Setting
RUL cap $R_{\max}$	125 cycles
Engine validation fraction	0.2 (20 engines held out)
Random split seed	42
Random forest trees	200
CNN window L	30
CNN optimizer	Adam, lr = 10^{-3}
CNN batch size	256
CNN early-stopping patience (val CNN)	12 epochs
CNN refit epochs (test CNN)	40 fixed epochs

B Hyperparameters used in experiments

References

- [1] A. Saxena, K. Goebel, D. Simon, and N. Eklund, Damage propagation modeling for aircraft engine run-to-failure simulation, in *Proc. Int. Conf. Prognostics and Health Management (PHM)*, 2008.
- [2] X. Li, Q. Ding, and J.-Q. Sun, Remaining useful life estimation in prognostics using deep convolution neural networks, *Reliability Engineering & System Safety*, 2018.
- [3] S. Zheng, K. Ristovski, A. Farahat, and C. Gupta, Long short-term memory network for remaining useful life prediction, in *Proc. IEEE PHM*, 2017.
- [4] F. O. Heimes, Recurrent neural networks for remaining useful life prediction, in *Proc. Int. Conf. Prognostics and Health Management (PHM)*, 2008.
- [5] R. Babu, G. Zhao, and X. Li, Deep convolutional neural network based regression approach for estimation of remaining useful life, in *International Conference on Database Systems for Advanced Applications*, Springer, 2016.
- [6] L. Breiman, Random forests, *Machine Learning*, 45(1):5–32, 2001.
- [7] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*, Chapman & Hall/CRC, 1994.